decision_missing_vs_skip

BOB-168 — DECISION: a listed-but-absent challenge blocks, in its own bucket

Revision: 2 **Last modified:** 2026-08-22T10:47:34Z **Status:** DECIDED and implemented. This file is the record; the runner's header cites it. Changing the exit contract in the script without changing this file leaves the next reader unable to tell a choice from an accident. **Mandate:** §11.4.3 (SKIP is topology-absent), §11.4.135 (absence blocks as FAIL does), §11.4.266 (advertised capability with no passing challenge), §11.4.248 (do not train readers to ignore red), §11.4.201(1)(6).

The question

The aggregator's exit code read only the FAIL count, so any number of missing entries exited 0. Should a listed-but-absent challenge participate in the exit code? Two defensible answers were on the table, plus a third path.

The decision

MISSING is a bucket of its own, separate from SKIP, and it blocks with exit 2. Precedence: `FAIL` (1) outranks `MISSING` (2) outranks success (0).

```
0  every listed entry ran and passed
1  at least one challenge FAILED         -> fix the code
2  at least one listed entry was MISSING -> fix the checkout
```

“MISSING” covers both *absent* and *present-but-not-executable*: in both cases the named entry attested nothing, because the runner refused it.

One accuracy note on the second case, added after review: the runner invokes via `bash "$script_path"`, which does **not** require the +x bit, so a mode-stripped entry could technically still execute. The `-x` gate predates this change and refusing remains the right conservative call — an entry whose mode says “not runnable” is a roster-integrity fact — but the printed reason must not claim more than it knows, so it reads “refused by the -x gate” rather than “cannot attest anything”.

Why — six reasons, in the order they actually decided it

1. SKIP and MISSING are different conditions and only one of them is licensed. §11.4.3’s SKIP-with-reason exists for a *topology-absent* precondition: a fact about the ENVIRONMENT, where the check is correct and there is genuinely nothing to assert. “The roster names a file that is not there” is a fact about the ROSTER. Conflating them let a roster-integrity failure borrow the legitimacy of a topology skip. That conflation — not the missing name — is the root defect, exactly as the task framing suspected.

2. At this seam there is no legitimate SKIP at all. This is what made the decision easy rather than balanced. Both skip branches in the aggregator (`! -f`, `! -x`) are roster-integrity failures. A challenge that legitimately skips for topology does so *inside its own script*, exits 0, and is counted PASS by the aggregator — it never reaches this bucket. So the SKIP bucket as written had **no legitimate members**. That removes the entire force of the “do not block” argument: blocking here cannot produce a runner that is red for a reason unrelated to the system under test, because no such reason can land in this bucket.

3. The measured consequence is not a corner case. Running the real runner where the challenges submodule is not checked out — an ordinary `git clone` without `--recursive` — produced `PASS: 0  FAIL: 0  SKIP: 16  TOTAL: 16`, **exit 0**. Sixteen challenges named, zero executed, caller told everything was fine. A blind instrument and a clean artifact returning the identical quiet zero is §11.4.201(6)’s FALSE-NULL verbatim, and it is §11.4.135’s absence-blocks-as-FAIL simply not applied here. The bank advertised coverage it did not deliver — the §11.4.266 shape — and nothing forced the question.

4. Why not fold MISSING into FAIL and exit 1. Because “a challenge failed” and “the bank could not be run” demand different responses — fix the code versus fix the checkout — and a caller that cannot distinguish them will chase the wrong one. It would also make the summary lie: `FAIL: 16` when nothing failed. Two exit codes keep both facts true.

5. Why exit 2 specifically, and not an invented number. This script already uses `exit 2` for precisely this class — its `BOBA_DURABLE` branch exits 2 when the durable helper file is not found, i.e. “the environment is not set up to run this”. The convention is the script’s own; it is reused, not minted (§11.4.6).

6. The §11.4.248 counter-argument, addressed rather than waved off. The real risk of a blocking rule is a permanently-red runner that trains readers to ignore red — the re-run-until-green trainer in another costume. That applies to red which is *flaky, irrelevant, or unactionable*. This red is none of those: it is deterministic, it names the exact entries, its remedy is one command (`git submodule update --init --recursive`), and it **cannot be re-run to green** — which is the property §11.4.248 actually wants. A green lie about an un-run bank is the more corrosive trainer of the two.

What was deliberately NOT done

- **SKIP was not deleted.** The counter and its summary line are retained at zero so a genuine topology-skip can be added later without re-merging the two conditions. The runner header says so, so the zero does not read as dead code.
- **`FAIL` semantics unchanged.** Exit 1 on a real failure behaves exactly as before; that is asserted by a dedicated case in the regression guard.
- **The glob-based `challenges/scripts/run_all_challenges.sh` was not touched.** It enumerates what exists, so it cannot have a dangling entry by construction and this defect class does not apply to it.

Guard

`tests/unit/test_run_all_challenges_missing_entry.sh` — drives the REAL runner (byte-identity asserted by sha256 inside the fixture, so it cannot decay into testing a lookalike), covers all-absent, not-executable, the §11.4.201(1) negative control (a complete healthy roster

must still exit 0), the unchanged FAIL path, **FAIL and MISSING simultaneously** (which pins the precedence documented above — without it, swapping the two exit blocks passed the suite), and a roster-drift check that fails if the fixture's list stops matching the runner's. Its fixture-setup path fails CLOSED: a bad root is a loud assertion failure with nothing executed, never a run against the invoker's cwd.